

Challuri Pruthagwin

2303A51951

10.08.2026

1. Running Sum of 1d Array

```
class Solution {
    public int[] runningSum(int[] nums) {
        for (int i = 1; i < nums.length; i++) {
            nums[i] = nums[i] + nums[i - 1];
        }
        return nums;
    }
}
```

2. Range Sum Query - Immutable

```
class NumArray {
    int[] prefix;
    public NumArray(int[] nums) {
        prefix = new int[nums.length + 1];
        for (int i = 0; i < nums.length; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }
    }
    public int sumRange(int left, int right) {
        return prefix[right + 1] - prefix[left];
    }
}
```

3. Find the Middle Index in Array

```
class Solution {
    public int findMiddleIndex(int[] nums) {
```

```
        int total = 0;
        for (int num : nums) {
            total += num;
        }
        int leftSum = 0;

        for (int i = 0; i < nums.length; i++) {
            int rightSum = total - leftSum - nums[i];
            if (leftSum == rightSum) {
                return i;
            }
            leftSum += nums[i];
        }
        return -1;
    }
}
```

4. Find Pivot Index

```
class Solution {
    public int pivotIndex(int[] nums) {
        int total = 0;
        for (int num : nums) {
            total += num;
        }
        int leftSum = 0;
        for (int i = 0; i < nums.length; i++) {
            int rightSum = total - leftSum - nums[i];
            if (leftSum == rightSum) {
                return i;
            }
            leftSum += nums[i];
        }
        return -1;
    }
}
```

```
}
```

5. Subarray Sum Equals K

```
class Solution {  
    public int subarraySum(int[] nums, int k) {  
        HashMap<Integer, Integer> map = new HashMap<>();  
  
        map.put(0, 1);  
        int sum = 0;  
        int count = 0;  
        for (int num : nums) {  
            sum += num;  
            if (map.containsKey(sum - k)) {  
                count += map.get(sum - k);  
            }  
            map.put(sum, map.getOrDefault(sum, 0) + 1);  
        }  
        return count;  
    }  
}
```

6. Contiguous Array

```
class Solution {  
    public int findMaxLength(int[] nums) {  
        HashMap<Integer, Integer> map = new HashMap<>();  
        map.put(0, -1);  
        int sum = 0;  
        int maxLength = 0;  
        for (int i = 0; i < nums.length; i++) {  
            if (nums[i] == 0) {  
                sum--;  
            } else {  
                sum++;  
            }  
        }  
    }  
}
```

```
if (map.containsKey(sum)) {
```

```
    maxLength = Math.max(maxLength, i -  
    map.get(sum));
```

```
    } else {
```

```
        map.put(sum, i);
```

```
    }
```

```
}
```

```
return maxLength;
```

```
}
```

```
}
```

7. Continuous Subarray Sum

```
class Solution {  
    public boolean checkSubarraySum(int[] nums, int k) {  
        HashMap<Integer, Integer> map = new HashMap<>();  
        map.put(0, -1);  
        int sum = 0;  
        for (int i = 0; i < nums.length; i++) {  
            sum += nums[i];  
            int remainder = sum % k;  
            if (map.containsKey(remainder)) {  
                if (i - map.get(remainder) >= 2) {  
                    return true;  
                }  
            }  
            else {  
                map.put(remainder, i);  
            }  
        }  
        return false;  
    }  
}
```

8. Matrix Block Sum

```
class Solution {
```

```
public int[][] matrixBlockSum(int[][] mat, int k) {
```

```
    int m = mat.length;
```

```
    int n = mat[0].length;
```

```
    int[][] prefix = new int[m + 1][n + 1];
```

```
    int[][] ans = new int[m][n];
```

```
    for (int i = 0; i < m; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            prefix[i + 1][j + 1] = mat[i][j]
```

```
                + prefix[i][j + 1]
```

```
                + prefix[i + 1][j]
```

```
                - prefix[i][j];
```

```
        }
```

```
    }
```

```
    for (int i = 0; i < m; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            int r1 = Math.max(0, i - k);
```

```
            int c1 = Math.max(0, j - k);
```

```
            int r2 = Math.min(m - 1, i + k);
```

```
            int c2 = Math.min(n - 1, j + k);
```

```
            ans[i][j] = prefix[r2 + 1][c2 + 1]
```

```
                - prefix[r1][c2 + 1]
```

```
                - prefix[r2 + 1][c1]
```

```
                + prefix[r1][c1];
```

```
        }
```

```
    }
```

```
    return ans;
```

```
}
```

```
}
```

9. Binary Search

```
class Solution {
```

```
    public int search(int[] nums, int target) {
```

```
        int left = 0;
```

```
        int right = nums.length - 1;
```

```
        while (left <= right) {
```

```
            int mid = left + (right - left) / 2;
```

```
            if (nums[mid] == target) {
```

```
                return mid;
```

```
            }
```

```
            if (nums[mid] < target) {
```

```
                left = mid + 1;
```

```
            } else {
```

```
                right = mid - 1;
```

```
            }
```

```
        }
```

```
        return -1;
```

```
    }
```

```
}
```

10. Search Insert Position

```
class Solution {
```

```
    public int searchInsert(int[] nums, int target) {
```

```
        int left = 0;
```

```
        int right = nums.length - 1;
```

```
        while (left <= right) {
```

```
            int mid = left + (right - left) / 2;
```

```
            if (nums[mid] == target) {
```

```
                return mid;
```

```
            }
```

```

        if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return left;
}

```

11. First Bad Version

```

public class Solution extends VersionControl {

    public int firstBadVersion(int n) {

        int left = 1;
        int right = n;

        while (left < right) {

            int mid = left + (right - left) / 2;

            if (isBadVersion(mid)) {

                right = mid;

            } else {

                left = mid + 1;

            }

        }

        return left;

    }
}

```

12. Sqrt(x)

```

class Solution {

    public int mySqrt(int x) {

        if (x < 2) {

```

```

            return x;

        }

        int left = 1;
        int right = x / 2;
        int ans = 1;

        while (left <= right) {

            int mid = left + (right - left) / 2;

            if (mid <= x / mid) {

                ans = mid;
                left = mid + 1;

            } else {

                right = mid - 1;

            }

        }

        return ans;

    }
}

```

13. Find First and Last Position of Element in Sorted Array

```

class Solution {

    public int[] searchRange(int[] nums, int target) {

        int first = findFirst(nums, target);
        int last = findLast(nums, target);

        return new int[]{first, last};

    }

    private int findFirst(int[] nums, int target) {

        int left = 0;

        int right = nums.length - 1;

        int ans = -1;

```

```

    }

while (left <= right) {
    int mid = left + (right - left) / 2;

    if (nums[mid] == target) {
        ans = mid;
        right = mid - 1;
    } else if (nums[mid] < target) {
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}

return ans;
}

private int findLast(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;
    int ans = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            ans = mid;
            left = mid + 1;
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
}

```

```

    }

    return ans;
}
}

```

14. Search in Rotated Sorted Array

```

class Solution {
    public int search(int[] nums, int target) {
        int left = 0;
        int right = nums.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] == target) {
                return mid;
            }

            if (nums[left] <= nums[mid]) {
                if (nums[left] <= target && target < nums[mid]) {
                    right = mid - 1;
                } else {
                    left = mid + 1;
                }
            } else {
                if (nums[mid] < target && target <= nums[right]) {
                    left = mid + 1;
                } else {
                    right = mid - 1;
                }
            }
        }
    }
}

```

```
        return -1;
    }
}
```

15. Find Peak Element

```
class Solution {
    public int findPeakElement(int[] nums) {
        int left = 0;
        int right = nums.length - 1;
        while (left < right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] < nums[mid + 1]) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        return left;
    }
}
```